

PRACTICE QUESTION PAPER 1: UNITS I AND II

Course: Programming in Python and Clean Code Principle

Coverage: Unit I and Unit II

Maximum suggested marks: 30

Section A: Very Short Answer Questions ($5 \times 1 = 5$)

Define clean code.

What is the return type of `input()`?

Write any two Python arithmetic operators.

What is the purpose of the `elif` statement?

What does the `break` statement do?

Section B: Short Answer Questions ($5 \times 2 = 10$)

Differentiate between a variable and a constant in Python.

Explain two characteristics of clean code.

Write a Python program to accept two numbers and print their sum.

Differentiate between `while` and `for` loops.

Explain `break`, `continue`, and `pass` with one line each.

Section C: Long Answer Questions ($3 \times 5 = 15$)

Explain Python data types, type conversion, `input()`, `print()`, and f-strings with suitable examples.

Explain `if`, `elif`, `else`, and nested-`if` statements. Write a program to display a grade based on marks.

Explain `for` loops, `while` loops, `range()`, and nested loops. Write a program to print a multiplication table.

PRACTICE QUESTION PAPER 2: UNITS III AND IV

Course: Programming in Python and Clean Code Principle

Coverage: Unit III and Unit IV

Maximum suggested marks: 30

Section A: Very Short Answer Questions ($5 \times 1 = 5$)

What is a function?

What is the purpose of the return statement?

Name any two Python data structures.

Define inheritance.

What does DRY stand for?

Section B: Short Answer Questions ($5 \times 2 = 10$)

Differentiate between parameters and arguments.

Explain the difference between a list and a tuple.

What is a dictionary? Give one example.

Explain encapsulation and abstraction.

Write any two SOLID principles and explain them briefly.

Section C: Long Answer Questions ($3 \times 5 = 15$)

Explain functions, default arguments, keyword arguments, return values, and recursion with examples.

Explain lists, tuples, sets, and dictionaries with their features and common operations.

Explain classes, objects, self, constructors, inheritance, and polymorphism with suitable Python programs.